Seaborn Quick Reference

Class notes cheat sheet: line, scatter, facet, histogram, KDE, count, bar, regression, pair and joint plots

A short, beginner-friendly summary of the Seaborn class notebook. The code is cleaned up and every example is checked to run. Each section follows the same order: what the plot does, syntax, examples with expected output, then notes.

- Run the code blocks from top to bottom in a Jupyter Notebook. Each example makes one plot.
- The images under the examples are your own plots, saved in the class notebook.
- Note, Common mistake and Correction boxes point out the points most worth remembering.
- Written for Seaborn 0.12 or newer. The `errorbar=` argument does not exist in older versions.

Contents

1. Setup and Data
2. Axes-Level vs Figure-Level
3. Line Plot
4. Scatter Plot
5. Facet Plot
6. Histogram
7. KDE Plot
8. Countplot
9. Bar Plot
10. Regplot and lmplot (Optional)
11. Pairplot
12. Joint Plot (Optional)
13. Parameter Summary

1. Setup and Data

1.1 Import the Libraries

Run these four lines once at the top of the notebook. The short names (aliases) are used in every example.

Library	Alias	Used for
`numpy`	`np`	Number functions, such as `np.median` and `np.max` (Section 9)
`pandas`	`pd`	Reading data and working with tables (DataFrames)
`matplotlib.pyplot`	`plt`	Seaborn draws on top of Matplotlib; `plt.show()` displays a plot
`seaborn`	`sns`	All the plots in this document

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

> **Note:** In Jupyter Notebook a plot appears automatically below the cell. In a normal `.py` script, add `plt.show()` after the plot.

1.2 Load a CSV File and Preview It

`pd.read_csv()` reads a CSV file into a DataFrame (a table). `.head()` shows the first 5 rows. Always check the column names before you plot.

Syntax and example (Dataset 1)

```
student = pd.read_csv('sns_data.csv')
student.head()
```

Expected output

student_id	class_level	subject	study_hours	test_score	week	attendance_rate	gender	hostel	Tshirt_size
1	Freshman	Math	5	78	1	90	Male	True	30
2	Freshman	Science	3	65	1	80	Female	False	20
3	Freshman	History	4	70	1	85	Male	True	20
4	Freshman	Language	2	60	1	75	Female	True	40
5	Sophomore	Math	6	82	1	92	Male	True	20

- Typing only the DataFrame name (for example `student`) shows the whole table. Long tables are shortened with `...` and the size is printed at the bottom, such as `[100 rows x 14 columns]`.
- Column names are **case-sensitive**: `Marks` and `Tshirt_size` must be typed exactly like this.

Common mistake: Writing `y='marks'` instead of `y='Marks'` gives `ValueError: Could not interpret value marks for y. An entry with this name does not appear in data.`

1.3 The Two Class Datasets

Your notebook uses two CSV files and stores **both** in the variable `student`. The second `read_csv` replaces the first, so reload the correct file whenever you switch. Each section below says which dataset it uses.

	Dataset 1: sns_data.csv	Dataset 2: student_dataset_complete.csv
Used in	Sections 3 and 4 (line, scatter)	Sections 5 to 12 (facet to joint plot)
Size	10 columns	100 rows, 14 columns
`gender`	`Male`, `Female`	`F`, `M`
`hostel`	`True`, `False`	`0`, `1`
`class_level`	Text: Freshman, Sophomore, Junior, Senior	Number: 10, 11, 12 (text names are in `class_name`)
`attendance_rate`	Percentage, e.g. `90`	Fraction from 0.80 to 1.00
`week`	1 to 6	1 to 5
`subject`	Math, Science, History, Language	Science, Math, English
`study_hours`, `test_score`	Whole numbers, e.g. `5` and `78`	Decimals, e.g. `4.508` and `19.202`
`Tshirt_size`	20, 30, 40	1 to 4

Note: In Dataset 2, `study_hours` has the same values as `time_study`, and `test_score` has the same values as `Marks` (in the rows shown in your notes). That is why `Marks` and `test_score` give similar plots.

Correction: Because `student` changes meaning halfway through the notebook, running an old cell after the reload can fail or give a different plot. Re-run the matching `read_csv` line first.

1.4 Built-in Datasets

Seaborn includes over 20 small practice datasets. `sns.load_dataset()` loads one by name. It needs an internet connection the first time.

Syntax and example

```
tips_data = sns.load_dataset("tips")
tips_data.head()
```

Expected output

total_bill	tip	sex	smoker	day	time	size
16.99	1.01	Female	No	Sun	Dinner	2
10.34	1.66	Male	No	Sun	Dinner	3
21.01	3.50	Male	No	Sun	Dinner	3
23.68	3.31	Male	No	Sun	Dinner	2
24.59	3.61	Female	No	Sun	Dinner	4

Tip: `sns.get_dataset_names()` returns the list of all available built-in datasets.

```
sns.get_dataset_names()
```

2. Axes-Level vs Figure-Level Functions

Every Seaborn plot function belongs to one of two groups. This idea is used in every section that follows.

- **Figure**: the entire canvas (window) where plots are displayed. It can hold one plot or many plots.
- **Axes**: one individual plotting area inside a Figure. It contains the actual graph, such as a scatter plot, histogram or line plot.

	Axes-level function	Figure-level function
Draws	One plot on one Axes	On its own Figure (one or many plots)
Returns	An `Axes` object	A `FacetGrid`, `PairGrid` or `JointGrid` object
Facets (`col=`, `row=`)	Not available	Available
Plot type is chosen by	The function name	The `kind=` argument (`relplot`, `displot`)

Which function makes which plot

Plot	Axes-level	Figure-level	Best for
Line	`lineplot`	`relplot(kind='line')`	How a value changes over time or order
Scatter	`scatterplot`	`relplot(kind='scatter')`	Relationship between two numbers
Histogram	`histplot`	`displot(kind='hist')`	How one number is spread out (counts in bins)
KDE	`kdeplot`	`displot(kind='kde')`	Smooth curve of how one number is spread out
Count	`countplot`	not covered in class	Number of rows in each category
Bar	`barplot`	not covered in class	Average (or other summary) for each category
Regression	`regplot`	`lmplot`	Scatter plot with a trend line
Pair	none	`pairplot`	Every pair of numeric columns at once
Joint	none	`jointplot`	Scatter plot with both distributions on the edges

What Jupyter prints above the plot

```
sns.lineplot(...)  ->  <Axes: xlabel='week', ylabel='attendance_rate'>
sns.relplot(...)   ->  <seaborn.axisgrid.FacetGrid at 0x251978feba0>
```

- These text lines are the returned object, not errors. The number after `0x` is different every run.

- Use an axes-level function for one simple plot. Use a figure-level function when you need `col=` or `row=` to split the data into several plots.
- `relplot` draws a scatter plot unless you give `kind='line'`. `displot` draws a histogram unless you give another `kind`.

Common mistake: Facet arguments only work with figure-level functions. `sns.scatterplot(..., col='gender')` fails with `AttributeError: PathCollection.set() got an unexpected keyword argument col`. Use `sns.relplot(kind='scatter', ..., col='gender')` instead.

Common mistake: Axes-level plots called one after another in the same cell are drawn on top of each other. Put `plt.show()` after each plot, as the multi-plot examples in this document do.

3. Line Plot

Dataset 1 (sns_data.csv) | Axes-level: sns.lineplot() | Figure-level: sns.relplot(kind='line')

A line plot shows how a number changes along an ordered x value, for example `attendance_rate` over each `week`.

Syntax

Template (replace df and the column names)

```
sns.lineplot(data=df, x='x_column', y='y_column',
             hue='group_column', errorbar=None)

sns.relplot(kind='line', data=df, x='x_column', y='y_column',
            hue='group_column', errorbar=None)
```

Examples

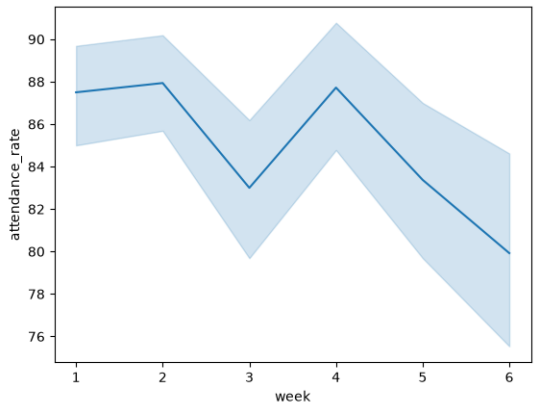
Example 1: Default line plot (axes-level)

The line is the mean `attendance_rate` of each week. The shaded band is the 95% confidence interval.

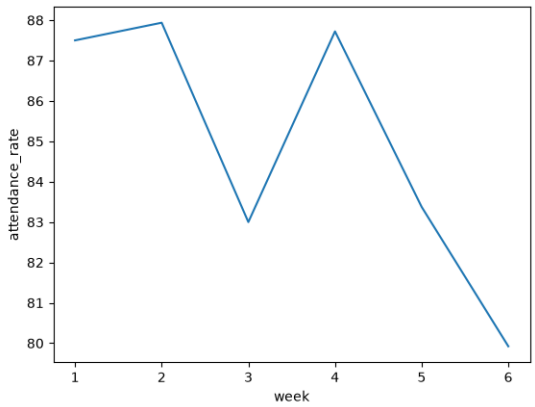
```
sns.lineplot(data=student, x='week', y='attendance_rate')
```

Example 2: Remove the shaded band

```
sns.lineplot(data=student, x='week', y='attendance_rate', errorbar=None)
```



Example 1: shaded band



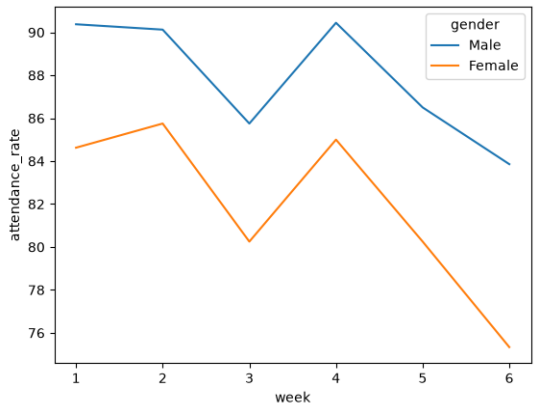
Example 2: errorbar=None

Example 3: One line for each group with hue

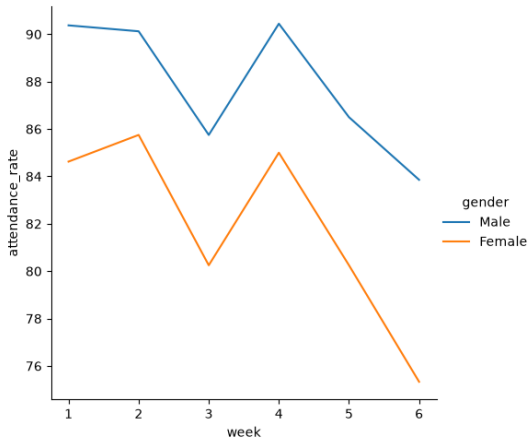
```
sns.lineplot(data=student, x='week', y='attendance_rate',
             errorbar=None, hue='gender')
```

Example 4: The same plot, figure-level

```
sns.relplot(kind='line', data=student, x='week', y='attendance_rate',
            errorbar=None, hue='gender')
```



Example 3: lineplot



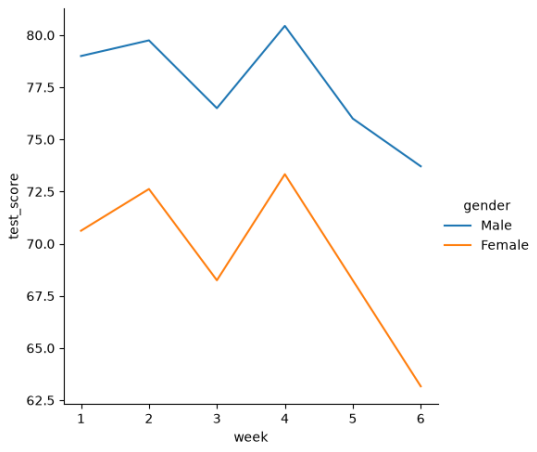
Example 4: relplot (legend outside)

Example 5: Other columns for y and hue

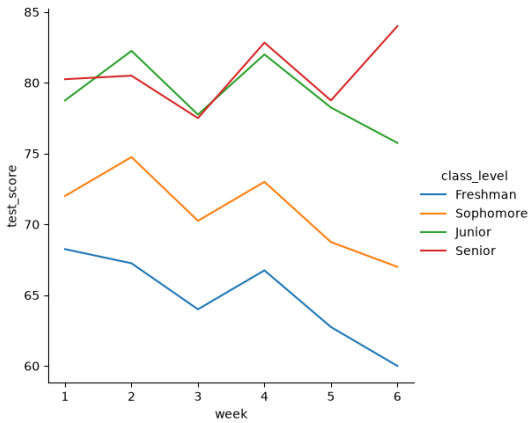
```
# test_score for each gender
sns.relplot(kind='line', data=student, x='week', y='test_score',
            errorbar=None, hue='gender')
plt.show()

# test_score for each class_level
sns.relplot(kind='line', data=student, x='week', y='test_score',
            errorbar=None, hue='class_level')
plt.show()

# attendance_rate for each class_level
sns.relplot(kind='line', data=student, x='week', y='attendance_rate',
            errorbar=None, hue='class_level')
plt.show()
```



test_score, hue='gender'



test_score, hue='class_level'

Notes and common mistakes

- When one week has many rows, Seaborn plots the **mean** and shades the 95% confidence interval. `errorbar=None` removes this band.
- `hue='gender'` draws one line per group, each with its own color and a legend.
- `x` holds the ordered values (week). `y` holds the number you are tracking.

Common mistake: Writing `sns.relplot(...)` without `kind='line'` draws a scatter plot, not a line plot.

> **Note:** Seaborn older than 0.12 uses `ci=None` instead of `errorbar=None`. In newer versions `ci=None` still runs but shows a FutureWarning.

4. Scatter Plot

Dataset 1 (sns_data.csv) and tips_data (Section 1.4) | Axes-level: sns.scatterplot() | Figure-level: sns.relplot(kind='scatter')

A scatter plot shows each row as one dot. It is used to see the relationship between two numeric columns, such as `study_hours` and `test_score`.

Syntax

Template (replace df and the column names)

```
sns.scatterplot(data=df, x='x_column', y='y_column',
                hue='column', style='column', size='column')

sns.relplot(kind='scatter', data=df, x='x_column', y='y_column',
            hue='column', style='column', size='column')
```

Parameter	Effect	Example from your notes
`hue`	Different **colors** for each group	`hue='gender'`
`style`	Different **marker shapes** for each group	`style='subject'`
`size`	Different **marker sizes** based on the column values	`size='Tshirt_size'`

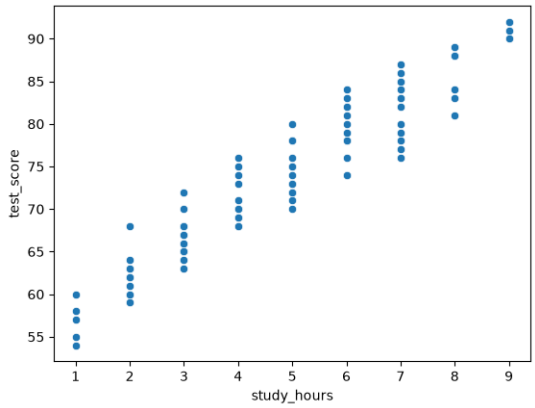
Examples

Example 1: Basic scatter plot

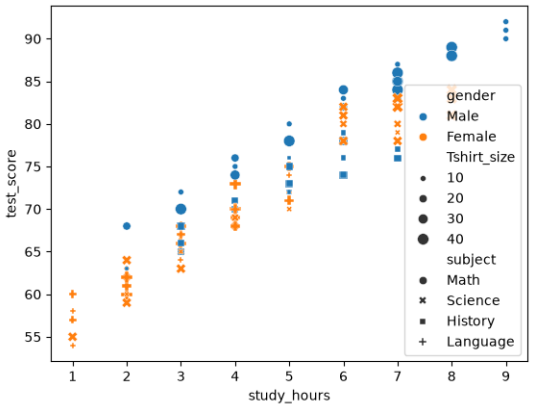
```
sns.scatterplot(data=student, x='study_hours', y='test_score')
```

Example 2: Add hue, style and size

```
# hue='gender'       -> a different color for each gender
# style='subject'    -> a different marker shape for each subject
# size='Tshirt_size' -> a different marker size for each Tshirt_size value
sns.scatterplot(data=student, x='study_hours', y='test_score',
                hue='gender', style='subject', size='Tshirt_size')
```



Example 1



Example 2

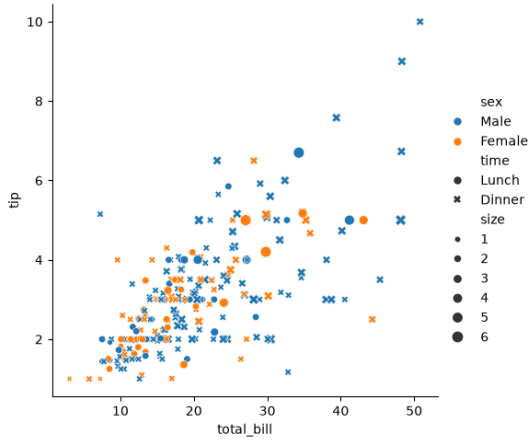
Example 3: The same plot, figure-level

```
sns.relplot(kind='scatter', data=student, x='study_hours', y='test_score',
            hue='gender', style='subject', size='Tshirt_size')
```

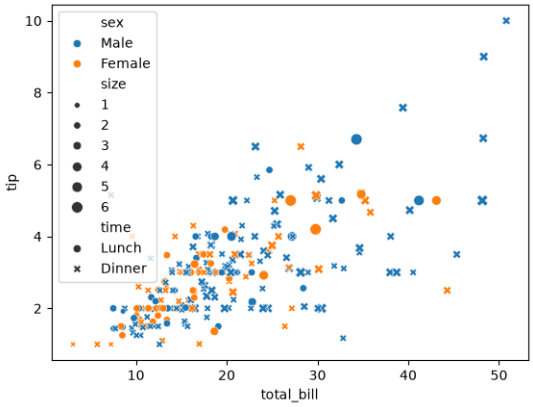
Example 4: Built-in tips dataset (relplot, then scatterplot)

```
# tips_data is loaded in Section 1.4
sns.relplot(kind='scatter', data=tips_data, x='total_bill', y='tip',
            hue='sex', style='time', size='size')
plt.show()

sns.scatterplot(data=tips_data, x='total_bill', y='tip',
                hue='sex', style='time', size='size')
```



relplot: legend outside the plot



scatterplot: legend inside, covers dots

Notes and common mistakes

- Use `hue`, `style` and `size` only when they add useful information. Too many make the plot hard to read.
- In `size='size'` the first `size` is the Seaborn parameter and the second is the column name in `tips_data`.
- `relplot` is a scatter plot by default, so `kind='scatter'` is optional. It is written in class to make the plot type clear.

> **Correction:** Your notebook ran Example 2 twice, with the comments split across two cells. It is shown once here with all three comments together.

5. Facet Plot

Dataset 2 (student_dataset_complete.csv) | Figure-level: sns.relplot() with col= and row=

A **Facet Plot** divides a dataset into several smaller plots (facets) based on one or more categorical columns. All small plots use the same axis ranges, so groups are easy to compare.

Syntax

Template (replace df and the column names)

```
sns.relplot(kind='scatter', data=df, x='x_column', y='y_column',
            col='column', row='column', col_wrap=2)
```

Examples

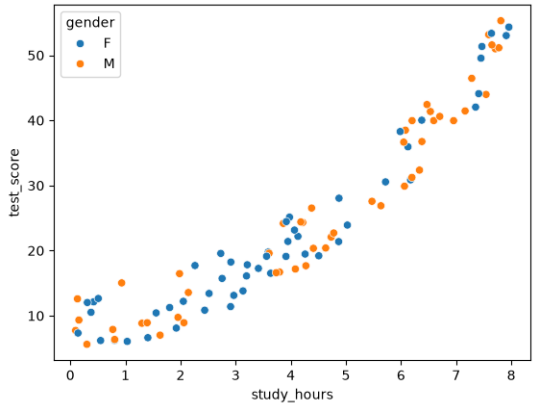
Example 1: Switch to Dataset 2

```
student = pd.read_csv('student_dataset_complete.csv')
student.head()
```

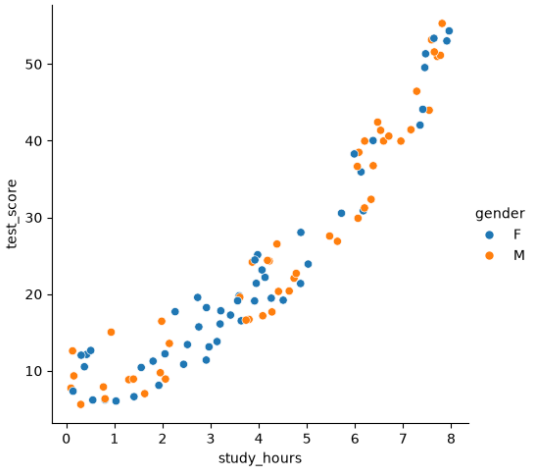
Example 2: One plot for all groups (not yet a facet plot)

```
sns.scatterplot(data=student, x='study_hours', y='test_score', hue='gender')
plt.show()

sns.relplot(kind='scatter', data=student, x='study_hours', y='test_score',
            hue='gender')
```



scatterplot with hue

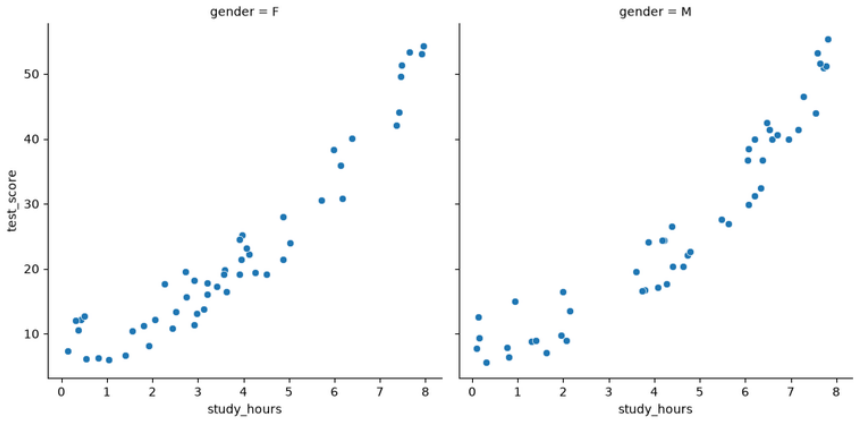


relplot with hue

Example 3: One plot per gender with col

`col='gender'` creates a separate plotting area for each gender. This is the facet plot.

```
sns.relplot(kind='scatter', data=student, x='study_hours', y='test_score',
            col='gender')
```



col='gender': one plot for F and one for M

Example 4: Rows and columns together

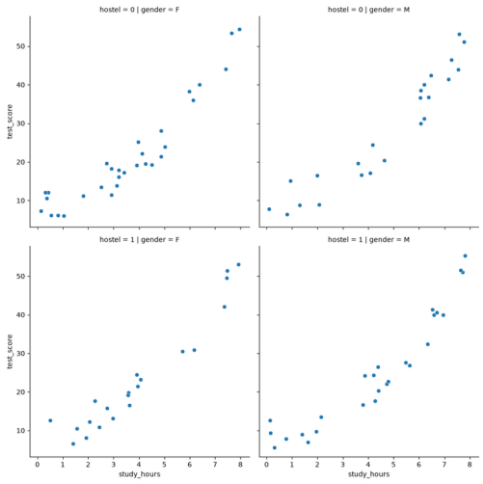
`col='gender'` makes one column for each gender. `row='hostel'` makes one row for each hostel value (0 and 1), giving a 2 × 2 grid.

```
sns.relplot(kind='scatter', data=student, x='study_hours', y='test_score',
            col='gender', row='hostel')
```

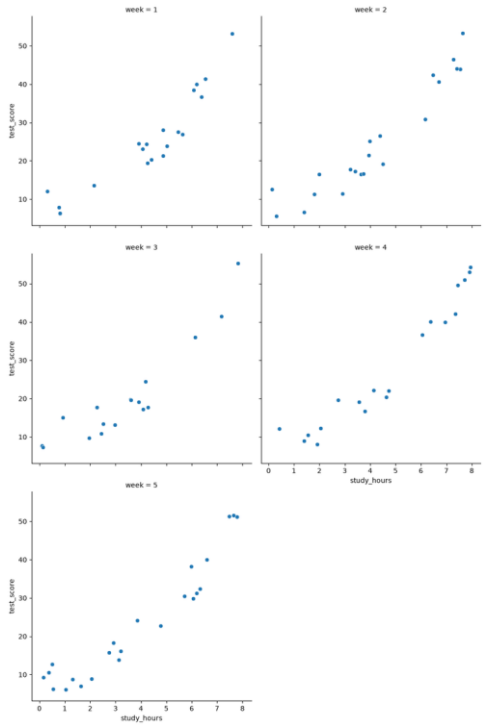
Example 5: Limit the plots per row with col_wrap

`col='week'` makes 5 plots. `col_wrap=2` allows at most 2 plots in a row, then continues on the next row.

```
sns.relplot(kind='scatter', data=student, x='study_hours', y='test_score',
            col='week', col_wrap=2)
```



Example 4: col and row



Example 5: col_wrap=2

Notes and common mistakes

- `scatterplot()` and `relplot(hue=...)` keep all groups in one plot. `relplot(col=...)` splits the groups into separate plots. That split is what makes it a facet plot.
- Each facet has a title above it, such as `hostel = 0 | gender = F`.

Common mistake: `col_wrap` cannot be used together with `row`. Seaborn stops with `ValueError: Cannot use row and col_wrap together.`

6. Histogram

Dataset 2 and tips_data | Axes-level: sns.histplot() | Figure-level: sns.displot(kind='hist')

A histogram shows how the values of **one** numeric column are spread out. The range is cut into bins and the bar height is the number of rows in each bin. Only `x` is needed; the y axis is automatically `Count`.

Syntax

Template (replace df and the column names)

```
sns.histplot(data=df, x='x_column', hue='group_column',
             bins=10, element='step')

sns.displot(kind='hist', data=df, x='x_column', col='group_column', bins=10)
```

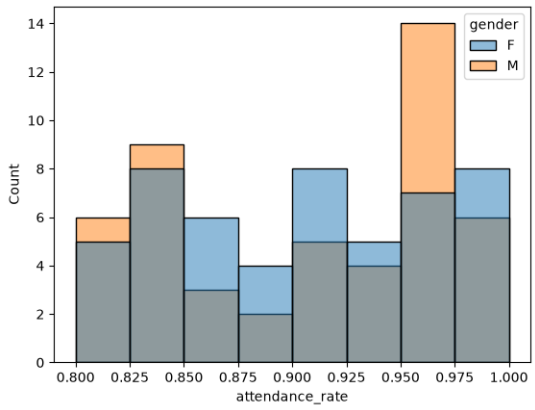
Examples

Example 1: Axes-level histogram with hue

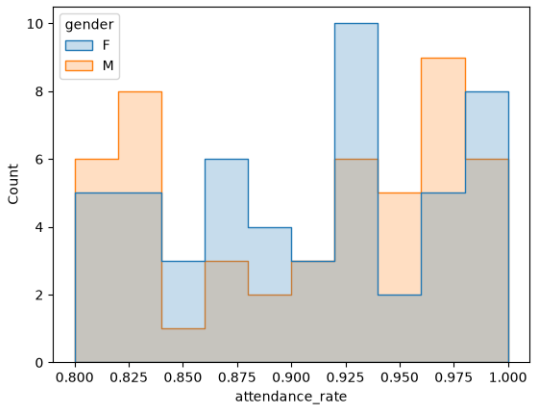
```
sns.histplot(data=student, x='attendance_rate', hue='gender')
```

Example 2: Set the number of bins and the step style

```
sns.histplot(data=student, x='attendance_rate', hue='gender',
             bins=10, element='step')
```



Example 1: default bars



Example 2: bins=10, element='step'

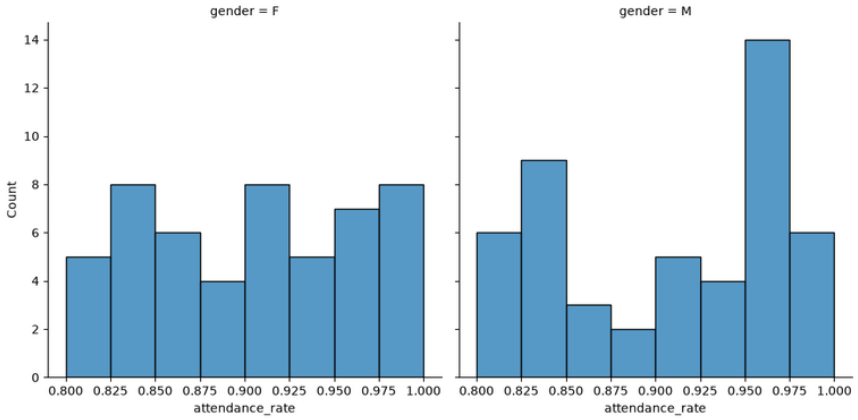
Example 3: Figure-level histogram

```
sns.displot(kind='hist', data=student, x='attendance_rate')
```

Example 4: One histogram for each gender

```
# attendance_rate
sns.displot(kind='hist', data=student, x='attendance_rate', col='gender')
plt.show()

# study_hours
sns.displot(kind='hist', data=student, x='study_hours', col='gender')
```



Example 4: attendance_rate, col='gender'

Example 5: Histograms of the tips data

```
# tips_data is loaded in Section 1.4
sns.displot(kind='hist', data=tips_data, x='total_bill', bins=20)
plt.show()

sns.displot(kind='hist', data=tips_data, x='tip', bins=20)
```

Notes and common mistakes

- `bins` is the number of bars. More bins show more detail but can look noisy; fewer bins look smoother.
- `element='step'` draws a stepped outline instead of solid bars (the default is `'bars'`). It makes overlapping groups easier to see.
- With `hue`, the groups are drawn on top of each other. Where they overlap, the colors mix.
- `displot` makes a histogram by default, so `kind='hist'` is optional. It is written in class to make the plot type clear.

7. KDE Plot

Dataset 2 and tips_data | Axes-level: sns.kdeplot() | Figure-level: sns.displot(kind='kde')

KDE (Kernel Density Estimate) draws a smooth curve that shows how the values of one numeric column are spread out. It is like a smoothed histogram. Higher parts of the curve mean values that occur more often. The y axis shows `Density`, not a count.

Syntax

Template (replace df and the column names)

```
sns.kdeplot(data=df, x='x_column', hue='group_column', fill=True)

sns.displot(kind='kde', data=df, x='x_column', col='group_column')
```

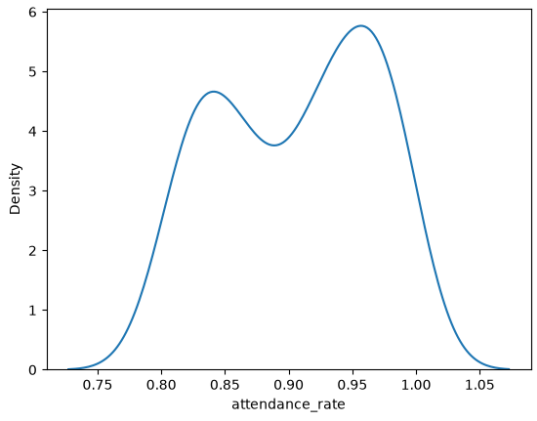
Examples

Example 1: Basic KDE plot (axes-level)

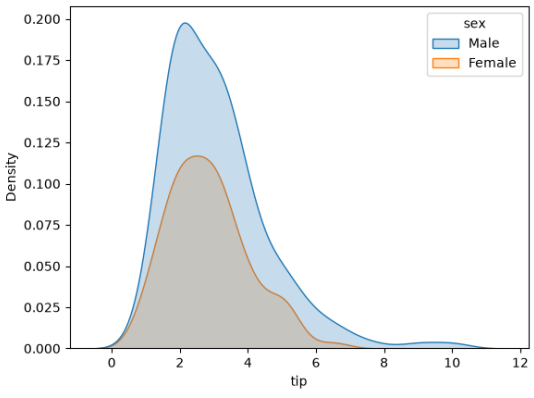
```
sns.kdeplot(data=student, x='attendance_rate')
```

Example 2: KDE with hue and shaded area

```
# tips_data is loaded in Section 1.4
sns.kdeplot(data=tips_data, x='tip', hue='sex', fill=True)
```



Example 1



Example 2: fill=True

Example 3: One KDE plot for each gender (figure-level)

```
sns.displot(kind='kde', data=student, x='attendance_rate', col='gender')
```

Notes and common mistakes

- `fill=True` shades the area under each curve.
- The curve can reach slightly beyond the smallest and largest values (for example above `1.0` for `attendance_rate`). This is normal because a KDE smooths the data.
- Use a histogram when you want exact counts. Use a KDE when you want to compare the shapes of groups.

8. Countplot

Dataset 2 | Axes-level: sns.countplot()

A countplot counts how many rows fall in each category of a column. The bar height is the count. Seaborn does the counting, so you give only one column.

Syntax

Template (replace df and the column names)

```
sns.countplot(data=df, x='category_column', hue='group_column')
```

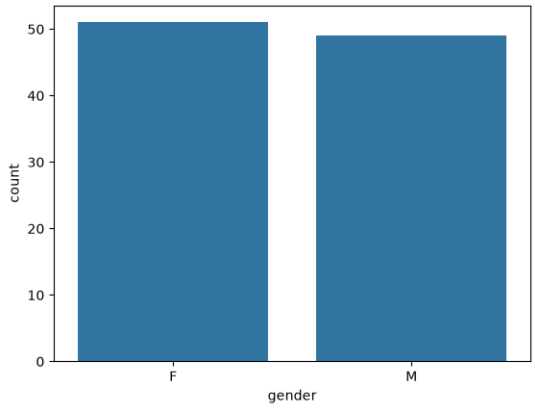
Examples

Example 1: Count of students by gender

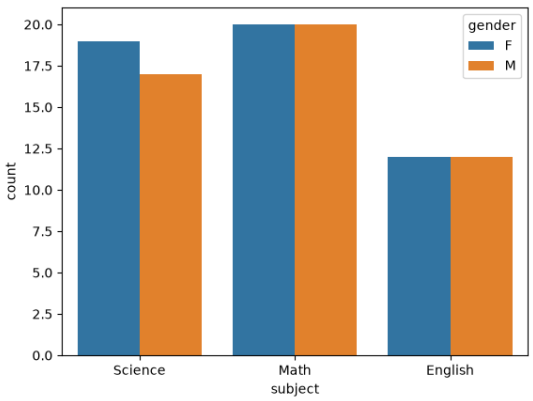
```
sns.countplot(data=student, x='gender')
```

Example 2: Count by subject, split by gender

```
sns.countplot(data=student, x='subject', hue='gender')
```



Example 1



Example 2

Notes and common mistakes

- Use a countplot for categorical columns such as `gender` or `subject`. For a numeric column, use a histogram.
- A countplot counts rows. A bar plot (Section 9) summarizes a numeric column.

> **Common mistake:** Do not give both `x` and `y`. `sns.countplot(data=student, x='gender', y='Marks')` fails with `TypeError: Cannot pass values for both x and y.`

9. Bar Plot

Dataset 2 | Axes-level: sns.barplot()

A bar plot draws one bar for each category. The bar height is a summary of a numeric column. By default this is the **mean**. The `estimator` argument changes the summary.

Syntax

Template (replace df and the column names)

```
sns.barplot(data=df, x='category_column', y='number_column',
            errorbar=None, estimator=np.median)
```

Examples

Example 1: Mean (the default)

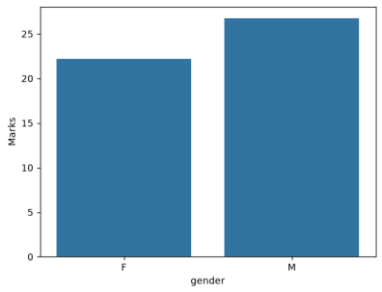
```
sns.barplot(data=student, x='gender', y='Marks', errorbar=None)
```

Example 2: Median

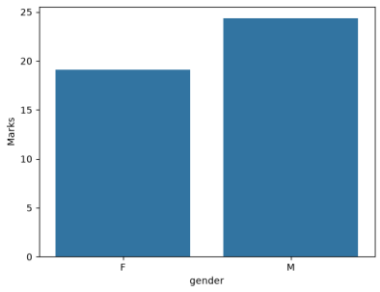
```
sns.barplot(data=student, x='gender', y='Marks', errorbar=None,
            estimator=np.median)
```

Example 3: Maximum

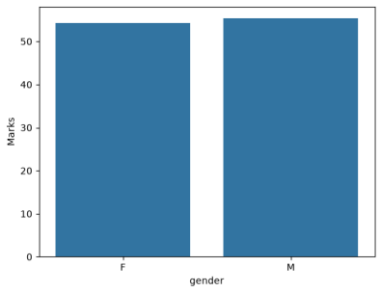
```
sns.barplot(data=student, x='gender', y='Marks', errorbar=None,
            estimator=np.max)
```



Example 1: mean



Example 2: median



Example 3: maximum

Check what the bars show with pandas

`groupby` splits the table by gender. The result is one number per gender, the same as the bar heights.

```
student.groupby('gender')['Marks'].mean()
```

Notes and common mistakes

- `errorbar=None` removes the black error bars (the 95% confidence interval) that `barplot` draws by default.
- Seaborn also accepts the summary name as text, for example `estimator='median'`.

> **Correction:** In your notebook the median plot had the comment `# mean`. The default is the mean, but `estimator=np.median` gives the **median**, so the comment was wrong. Your unused line `student_ = student.groupby('gender')` is replaced by the working check above.

10. Regplot and lmplot (Optional)

Dataset 2 | Axes-level: sns.regplot() | Figure-level: sns.lmplot()

These draw a scatter plot **plus** a regression line (the best-fit straight line) and a shaded band around it. They show the trend between two numeric columns.

Syntax

Template (replace df and the column names)

```
sns.regplot(data=df, x='x_column', y='y_column')

sns.lmplot(data=df, x='x_column', y='y_column', hue='group_column')
```

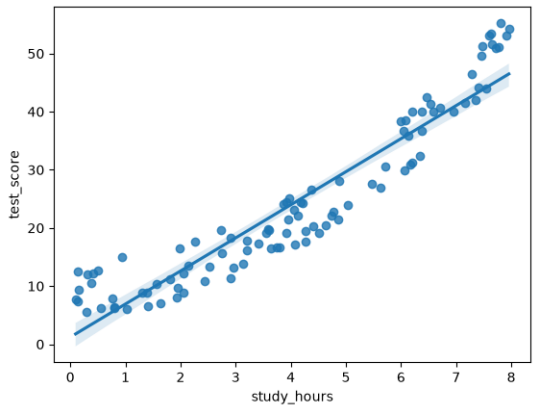
Examples

Example 1: One regression line (axes-level)

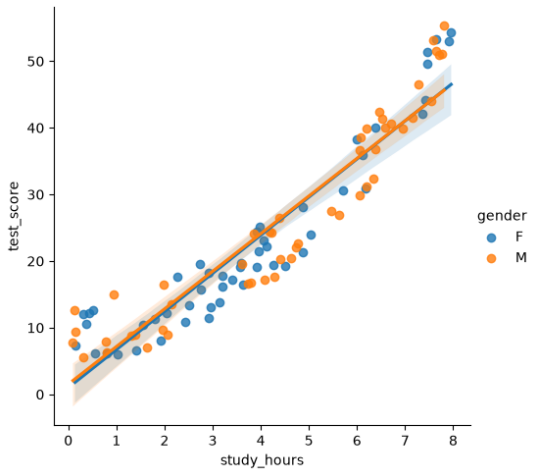
```
sns.regplot(data=student, x='study_hours', y='test_score')
```

Example 2: One regression line for each gender (figure-level)

```
sns.lmplot(data=student, x='study_hours', y='test_score', hue='gender')
```



Example 1: regplot



Example 2: lmplot with hue

Notes and common mistakes

- A line that goes up means that when `study_hours` increases, `test_score` also tends to increase.
- Both `x` and `y` must be numeric columns.

Common mistake: `regplot` has no `hue` argument. `sns.regplot(..., hue='gender')` fails with `TypeError: regplot() got an unexpected keyword argument hue`. Use `lmplot` for separate lines per group.

11. Pairplot

Dataset 2 | Figure-level: sns.pairplot()

A pairplot makes a grid of plots that compares **every pair** of numeric columns. The diagonal shows the distribution of each column on its own.

Syntax

Template (replace df and the column names)

```
sns.pairplot(data=df, hue='group_column', kind='hist')
```

Examples

Example 1: Choose the columns first

Double square brackets `[[ ]]` select several columns and create a smaller DataFrame. A pairplot of every column would make a huge grid, so pick only the columns you need.

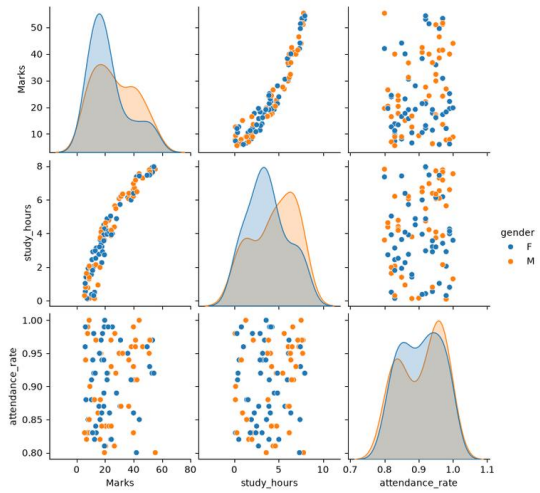
```
student_marks = student[['Marks', 'study_hours', 'attendance_rate', 'gender']]
student_marks
```

Example 2: Pairplot with hue

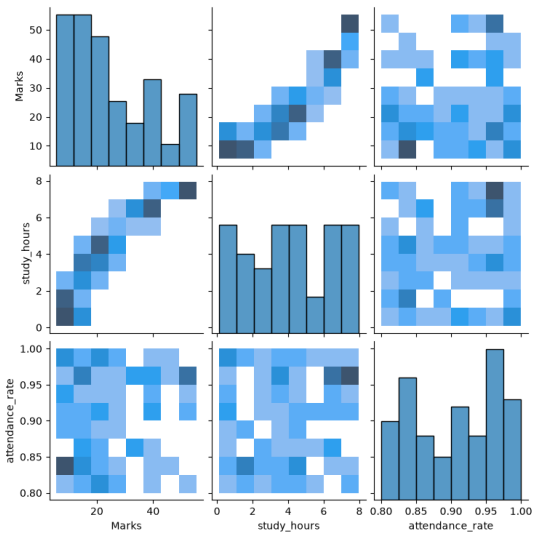
```
sns.pairplot(data=student_marks, hue='gender')
```

Example 3: Pairplot with kind='hist'

```
sns.pairplot(data=student_marks, kind='hist')
```



Example 2: hue='gender'



Example 3: kind='hist'

Notes and common mistakes

- `student_marks` has 100 rows and 4 columns. Three of them are numeric, so the grid is 3 × 3.
- `gender` is text, so it gets no axis. It is used only for `hue`.
- The default plots are scatter plots. With `hue` the diagonal shows KDE curves; without `hue` it shows histograms.
- `kind='hist'` replaces the scatter plots with 2-D histograms. Darker squares mean more rows.

12. Joint Plot (Optional)

Dataset 2 (uses student_marks from Section 11) | Figure-level: sns.jointplot()

A joint plot shows a scatter plot of two numeric columns in the center, with the distribution of each column on the top and right edges.

Syntax

Template (replace df and the column names)

```
sns.jointplot(data=df, x='x_column', y='y_column',
              hue='group_column', kind='kde')
```

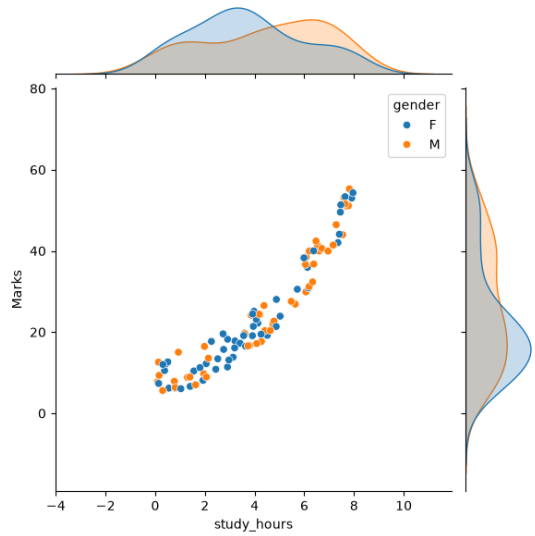
Examples

Example 1: Scatter joint plot with hue

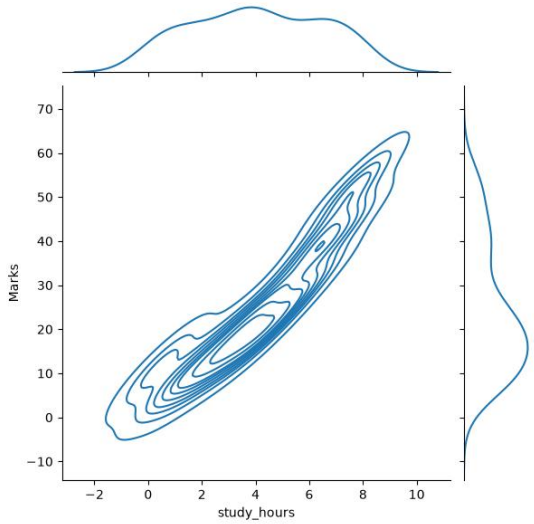
```
sns.jointplot(data=student_marks, x='study_hours', y='Marks', hue='gender')
```

Example 2: KDE joint plot

```
sns.jointplot(data=student_marks, x='study_hours', y='Marks', kind='kde')
```



Example 1: scatter with hue



Example 2: kind='kde'

Notes and common mistakes

- The default is `kind='scatter'`.
- `kind='kde'` draws density contour lines in the center. Lines packed closely together mean many rows in that area. The edges show KDE curves.

13. Parameter Summary

The arguments used in this document, in one place.

Parameter	What it does	Used in
`data=`	The DataFrame that holds the columns	All functions
`x=`, `y=`	Columns for the horizontal and vertical axes. `histplot`, `kdeplot` and `countplot` need only `x`	All functions
`hue=`	Colors the data by the groups in a column	Line, scatter, histogram, KDE, count, lmplot, pairplot, jointplot
`style=`	A different marker shape for each group	Scatter plot
`size=`	Marker size changes with the column values	Scatter plot
`col=`, `row=`	One small plot (facet) for each group, in columns or in rows	`relplot`, `displot`
`col_wrap=`	Maximum number of plots in one row. Use with `col=`, never with `row=`	`relplot`
`kind=`	Selects the plot type	`relplot` (line, scatter), `displot` (hist, kde), `pairplot` (hist), `jointplot` (kde)
`errorbar=None`	Removes the shaded band or error bars	`lineplot`, `relplot(kind='line')`, `barplot`
`estimator=`	Function that summarizes each bar (default mean)	`barplot`
`bins=`	Number of bars in the histogram	`histplot`, `displot(kind='hist')`
`element='step'`	Stepped outline instead of solid bars	`histplot`
`fill=True`	Shades the area under the curve	`kdeplot`